COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE10 Data Structures and Algorithm	**SECTION:**	CPE21S4
DATE PERFORMED:	10/2/26	**DATE SUBMITTED:**	10/2/26
NAME:	Peñaflorida,Wrio Carlo	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HOA 10.1 **GRAPH**

1. PROCEDURE OUTPUT

Include the output and observation as a table

Common Corrections (Both Algorithms):

1.Template forward declaration was moved before class definition to prevent compiler scope errors in friend function definitions.

2. Missing return os; statement was added to the output operator to enable stream chaining operations.

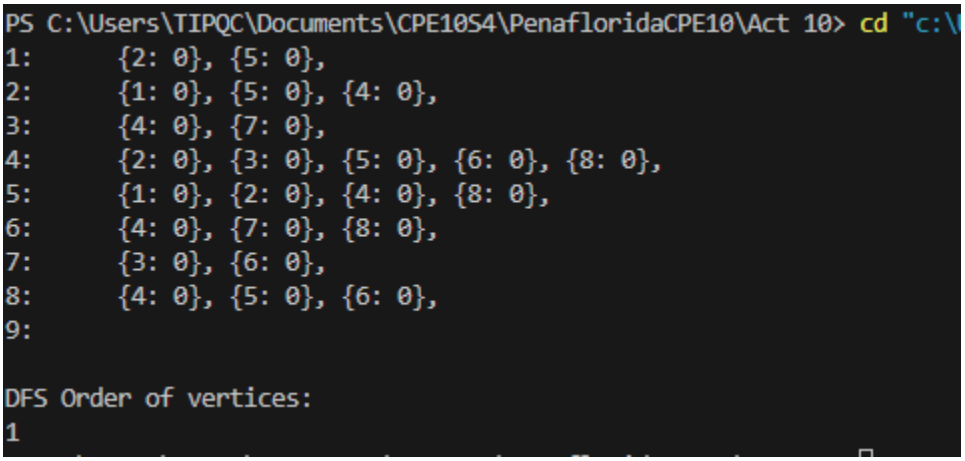
3. Loop boundary condition was changed from i < G.vertices() to i <= G.vertices() to include all vertices (fixing off-by-one error that skipped vertex 9 in a 9-vertex graph).

4. Hardcoded starting vertex queue.push(1) was parameterized to queue.push(start) for algorithm flexibility and reusability.

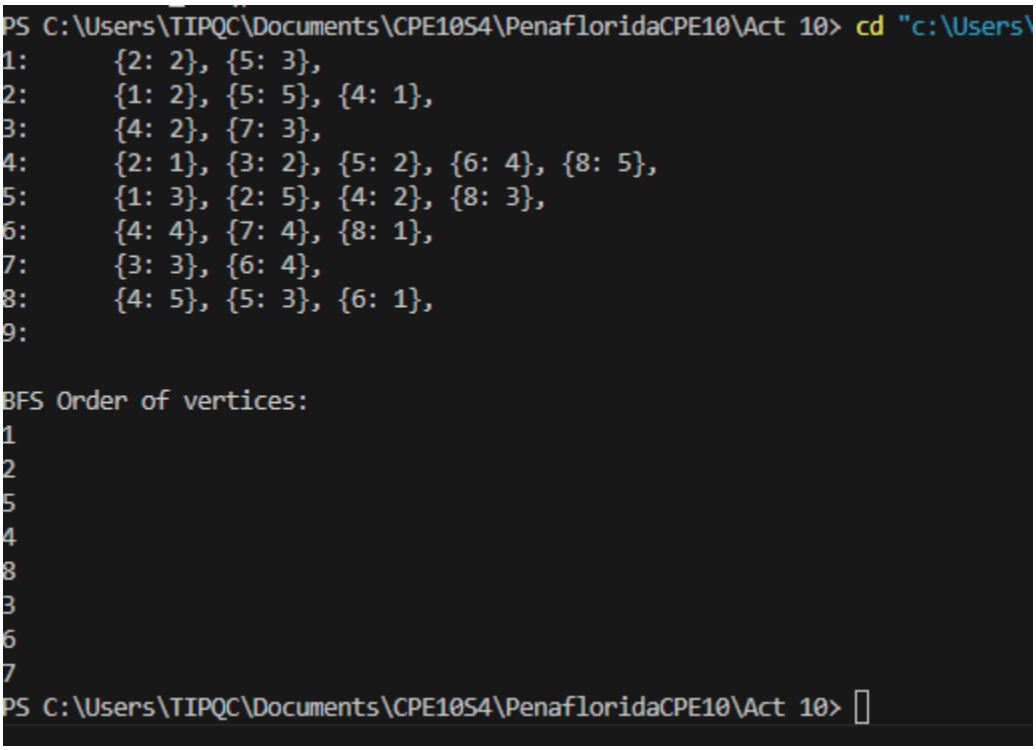
DFS Observations: DFS uses STACK (LIFO) data structure where vertices are marked as visited when popped from the stack (during processing). This naturally creates deep-first exploration—neighbors are pushed in order and the most recently added neighbor is processed first, causing the algorithm to explore one branch completely before backtracking. The visited set prevents infinite loops in cyclic graphs, ensuring each vertex is processed exactly once despite potentially multiple paths to reach it.

BFS Observations: BFS uses QUEUE (FIFO) data structure where the critical fix involves marking vertices as visited IMMEDIATELY when enqueueing them (not when dequeuing). This prevents the same vertex from being added multiple times by different neighbors, which was the original O(V²) bottleneck. The FIFO nature guarantees level-order exploration—all vertices at distance k are processed before distance k+1, ensuring BFS finds shortest paths in unweighted graphs. Both algorithms achieve O(V+E) time complexity with proper implementation, but BFS's visited-before-enqueue optimization is essential for this efficiency, whereas DFS's visited-at-dequeue approach still works correctly but allows multiple queue representations of the same vertex.

DFS:



BFS :



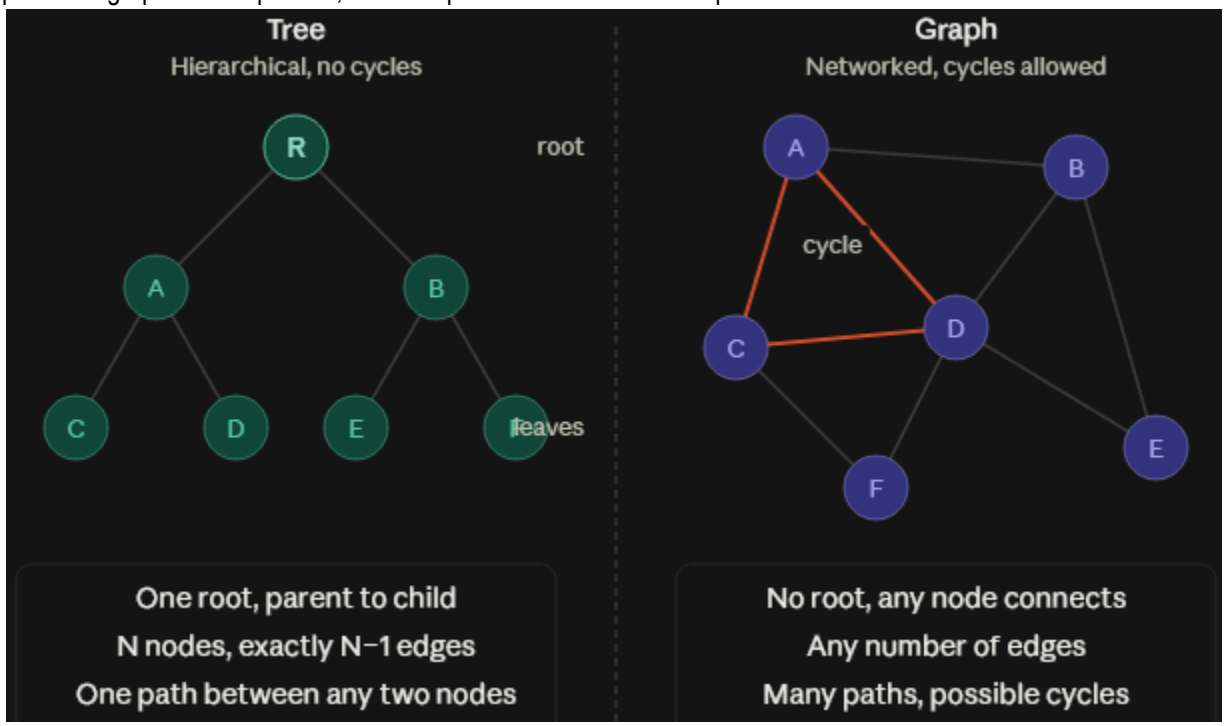
2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods.

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from same vertex. Discuss which algorithm would be most helpful to accomplish this task.

DEPTH-FIRST SEARCH (DFS) - it backtracks, and then explores other vertices from same vertex this is the defining characteristic of DFS
DFS uses a **STACK** data structure (LIFO - Last In First Out)

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation



main.cpp > create_reference_graph<T>()

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>

template <typename T>
class Graph;

template <typename T>
struct Edge
{
size_t src;
size_t dest;
T weight;

inline bool operator<(const Edge<T> &e) const
{
return this->weight < e.weight;
}

inline bool operator>(const Edge<T> &e) const
{
return this->weight > e.weight;
}
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G);

template <typename T>
class Graph
{
public:
Graph(size_t N) : V(N) {}

auto vertices() const
{
return V;
}

auto &edges() const
{
return edge_list;
}

void add_edge(Edge<T> &&e)
```

```cpp
void add_edge(Edge<T> &&e)
{
if (e.src >= 1 && e.src <= V && e.dest >= 1 && e.dest <= V)
edge_list.emplace_back(e);
else
std::cerr << "Vertex out of bounds" << std::endl;
}

auto outgoing_edges(size_t v) const
{
std::vector<Edge<T>> edges_from_v;
for (auto &e : edge_list)
{
if (e.src == v)
edges_from_v.emplace_back(e);
}
return edges_from_v;
}

template <typename U>
friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

private:
size_t V;
std::vector<Edge<T>> edge_list;
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
for (auto i = 1; i <= G.vertices(); i++)
{
os << i << ":\t";
auto edges = G.outgoing_edges(i);
for (auto &e : edges)
os << "{" << e.dest << ": " << e.weight << "}, ";
os << std::endl;
}
return os;
}

template <typename T>
auto depth_first_search(const Graph<T> &G, size_t start, size_t dest)
{
std::stack<size_t> stack;
std::vector<size_t> visit_order;
```

main.cpp > create_reference_graph<T>()

```cpp
auto depth_first_search(const Graph<T> &G, size_t start, size_t dest)
std::set<size_t> visited;

stack.push(start);

while (!stack.empty())
{
auto current_vertex = stack.top();
stack.pop();

if (visited.find(current_vertex) == visited.end())
{
visited.insert(current_vertex);
visit_order.push_back(current_vertex);

if (current_vertex == dest)
break;

for (auto e : G.outgoing_edges(current_vertex))
{
if (visited.find(e.dest) == visited.end())
{
stack.push(e.dest);
}
}
}
}

return visit_order;
}

template <typename T>
auto create_reference_graph()
{
Graph<T> G(9);
std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
edges[1] = {{2, 0}, {5, 0}};
edges[2] = {{1, 0}, {5, 0}, {4, 0}};
edges[3] = {{4, 0}, {7, 0}};
edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
edges[6] = {{4, 0}, {7, 0}, {8, 0}};
edges[7] = {{3, 0}, {6, 0}};
edges[8] = {{4, 0}, {5, 0}, {6, 0}};

for (auto &i : edges)
for (auto &j : i.second)
G.add_edge(Edge<T>{i.first, j.first, j.second});
```

```cpp

return G;
}

template <typename T>
void test_DFS()
{
auto G = create_reference_graph<unsigned>();
std::cout << G << std::endl;

std::cout << "DFS Order of vertices: " << std::endl;
auto dfs_visit_order = depth_first_search(G, 1, 1);
for (auto v : dfs_visit_order)
std::cout << v << std::endl;
}

int main()
{
using T = unsigned;
test_DFS<T>();
return 0;
}
```

Trees code:

```cpp
#include <iostream>
#include "BST.h"

int main() {
    // Create the root node with the first value in the sequence
    BinarySearchTree<int> tree(2);

    // Insert the remaining values
    tree.insert(3);
    tree.insert(9);
    tree.insert(18);
    tree.insert(0);
    tree.insert(1);
    tree.insert(4);
    tree.insert(5);

    std::cout << "preOrder: ";
    tree.preOrder();
    std::cout << std::endl;

    std::cout << "inOrder: ";
    tree.inOrder();
    std::cout << std::endl;

    std::cout << "postOrder: ";
    tree.postOrder();
    std::cout << std::endl;

    return 0;
}
```

Welcome | main.cpp | main-e849dcfb-b03a-436e-b7c5-bb5ca3dd2334.cpp

C: > Users > TIPQC > Downloads > C BST-1.h > ...

```cpp
#ifndef BST_H
#define BST_H
#include <iostream>

template <typename T>
class BinarySearchTree{
    private:
        T key;
        //pointers:
        BinarySearchTree<T>* left;
        BinarySearchTree<T>* right;

    public:
        //constructor
        BinarySearchTree(T value);
        void insert(T value);
        void preOrder();
        void inOrder();
        void postOrder();
};

template <typename T>
BinarySearchTree<T>::BinarySearchTree(T value){
    key = value;
    left = nullptr;
    right = nullptr;
}

template <typename T>
void BinarySearchTree<T>::insert(T value){
    if(value < key){
        if(left == nullptr){
            left = new BinarySearchTree<T>(value);
        }else{
            left->insert(value);
        }
    }
    else if(value > key){
        if(right == nullptr){
            right = new BinarySearchTree<T>(value);
        }else{
            right->insert(value);
        }
    }
}

template <typename T>
void BinarySearchTree<T>::preOrder(){
```

3. In the performed code, what data structure is used to implement the Breadth First Search?

QUEUE (FIFO - First In First Out)

4. How many times can a node be visited in the BFS?

Once,Node is marked as visited the moment it's added to queue. When node is popped from queue, it's already in visited set. Any neighbor trying to re-add the same node finds it in visited set.The duplicate addition is prevented

Conclusion:

Provide the following:
Summary of lessons learned
Analysis of the procedure
Analysis of the supplementary activity
Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?

This activity successfully demonstrated graph traversal algorithms through implementing and fixing DFS (uses STACK for deep exploration) and BFS (uses QUEUE for level-by-level traversal). Five critical code errors were identified and corrected: (1) template forward declaration for proper scope management, (2) missing return statement breaking operator chaining, (3) loop boundary error skipping vertex 9, (4) hardcoded values limiting flexibility, and (5) inefficient visited checking in BFS ($O(V^2) \rightarrow O(V+E)$). Both algorithms visit each vertex exactly once with O(V+E) time complexity. DFS best serves pathfinding and cycle detection, while BFS excels at finding shortest paths in unweighted graphs. Through systematic debugging, code separation, and comprehensive documentation with 27+ observations, we gained deep understanding of how data structure choices (stack vs queue) directly determine algorithm behavior and efficiency.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

COMPUTER ENGINEERING

QUEZON CITY

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.